

- 🏠 Home
- 🔖 Library
- 👤 Profile
- 📄 Stories
- 📊 Stats

- 👤 Following
- 🧠 Dev Genius
- 🐍 Python in Plain En...
- 🤖 Generative AI
- 📱 Realworld AI Use C...
- 📰 The Useful Tech
- ∞ Level Up Coding
- 🧠 AI Mind
- 👤 Deep concept
- 🐍 Top Python Librari...
- 📱 The Mac Alchemist
- ⌵ More

Data Science Collective

★ Member-only story

Building a Multi-Agent System That Turns One Sentence Into a \$500 Online Course

Curriculum, lectures, slides, quizzes, assignments — four LangGraph agents, one anti-slop reviewer, one afternoon of work.

 Anubhav

Follow

 14 min read · Apr 26, 2026

👏 132

💬

↺↻

🔖

▶

📄

⋮



Read the article for free [here](#).

There is one line in the input box: *Teach me how to build production-grade retrieval systems for LLM apps.*

Ten minutes later, the pipeline returns a directory:

```
course/
├── syllabus.md
├── lectures/
│   ├── 01_what_retrieval_actually_does.md
│   ├── 02_chunking_strategies_that_dont_ruin_recall.md
│   ├── 03_embedding_models_that_arent_oai_ada_002.md
│   ├── 04_vector_stores_and_when_they_are_overkill.md
│   ├── 05_hybrid_search_bm25_and_dense_together.md
│   ├── 06_rerankers_and_what_they_fix.md
│   ├── 07_query_rewriting_and_when_to_skip_it.md
│   ├── 08_evaluation_beyond_vibe_check.md
│   ├── 09_grounding_and_citation_discipline.md
│   ├── 10_freshness_and_re_indexing.md
│   ├── 11_latency_and_cost_tuning.md
│   └── 12_failure_modes_and_how_to_ship_with_them.md
└── quizzes/
    ├── module_1.json
    └── module_2.json
```

```
├── module_3.json
├── assignments/
│   ├── module_2_build_a_reranker.md
│   └── module_3_ship_a_grounded_answer_pipeline.md
├── slides/
└── course.pdf
```

The syllabus looks like this for the first module:

```
Module 1: Retrieval without the Buzzwords (3 lessons)
Lesson 1: What retrieval actually does
Objectives:
- Explain what the retrieval step is for (not what e
- Distinguish retrieval from generation and from ran
- Identify where retrieval belongs in the RAG pipeli
```

The receipt for this output is about \$0.26 in API spend and 14 minutes of wall-clock time. According to the 2026 pricing guides from [iSpring](#), a Level 1 custom eLearning module — narrated slides with static graphics and simple knowledge checks — costs between \$250 and \$500 per finished hour.

This pipeline does not ship a finished \$500 course on its own. It ships a first draft fast enough that an editor can finish it in an afternoon.

This post covers how to build that pipeline. We will use four agents in LangGraph and one anti-slop reviewer. We will set up one human approval gate between planning

and writing. We will follow one running example all the way through to the real PDF, and look closely at the exact failure modes this architecture prevents.

Why course generation is a pipeline and not a prompt

Most developers try to build course generators by writing one large prompt. They tell a single language model to write a full course on a topic. The syllabus looks fine and the first module is decent. By module three the text starts to drift. By module four the model contradicts module one. The tone wanders and the learning objectives quietly disappear.

No single context window is both long enough to hold a whole course and focused enough to make each lesson land.

Course generation is almost perfectly shaped for multi-agent decomposition. The stages are cleanly separable. Syllabus, lectures, assessments, and production map directly to real instructional design phases. The **ADDIE** framework (Analyze, Design, Develop, Implement, Evaluate) has used these exact seams for a long time.

Each phase has a different optimal model. You need a reasoning-heavy model for the syllabus and a cheap and fast model for the lecture body prose. You also need deterministic code for slide production.

The Instructional Agents paper (Yao et al., Arizona State University, EACL 2026) proved this works. They evaluated a multi-agent framework across five university-level computer science courses using human experts. Their Full Co-Pilot mode consistently outperformed autonomous generation.

We are going to build a production-oriented adaptation of that architecture.

The architecture

Our system uses four agents, one reviewer, and one human approval gate.

The Curriculum agent takes one input sentence and outputs a structured syllabus. The Content agent takes one lesson outline and outputs one full lecture script. This runs in parallel across all lessons. The Assessment agent takes one module and outputs a quiz and an assignment. The Production agent is deterministic Python. It takes all outputs and assembles the slide deck and PDF. The Anti-slop Reviewer is called after every content generation. It returns structured corrections or an approval. We use a relatively cheaper model for this task.

A supervisor node wires them together. We use explicit routing for the deterministic phase transitions and

dynamic parallel fan-out for the writing phase.

Shared State and the Data Contracts

Every agent reads and writes to a shared state. The state schema is the spine of the pipeline. We use Pydantic models to define the data contracts.

```
from typing import Annotated, Literal, TypedDict
from pydantic import BaseModel, Field

BloomLevel = Literal[
    "remember", "understand", "apply", "analyze", "evaluate"
]

class LearningObjective(BaseModel):
    id: str = Field(description="short stable id, e.g. 'lo")
    text: str = Field(description="measurable, learner-cen")
    bloom: BloomLevel
```

```
module_id: str

class Lesson(BaseModel):
    id: str
    module_id: str
    title: str
    objectives: list[str] = Field(description="LearningObj")
    prerequisites: list[str] = Field(default_factory=list)
    outline: list[str] = Field(description="hook, concept,")

class Module(BaseModel):
    id: str
    title: str
    summary: str
    lessons: list[str]

class Syllabus(BaseModel):
    topic: str
    audience: str
    prerequisites: list[str]
    modules: list[Module]
    lessons: list[Lesson]
    objectives: list[LearningObjective]

class Lecture(BaseModel):
    lesson_id: str
    title: str
    body_markdown: str
    checklist: list[str]
    rewrite_count: int = 0

class ReviewIssue(BaseModel):
    category: Literal[
        "filler_phrase", "buzzword", "missing_specificity",
        "uniform_rhythm", "vague_conclusion", "lexical_rep
    ]
    example: str = Field(description="literal quote from t
    fix: str = Field(description="one-sentence correction

class ReviewVerdict(BaseModel):
    lesson_id: str
    approved: bool
    severity: Literal["low", "medium", "high"] = "low"
    issues: list[ReviewIssue] = Field(default_factory=list

class Quiz(BaseModel):
    module_id: str
    questions: list[dict]
```

```
class Assignment(BaseModel):
    module_id: str
    brief_markdown: str
    rubric: list[str]

def merge_dicts(left: dict, right: dict) -> dict:
    return {**left, **right}

class CourseState(TypedDict, total=False):
    topic: str
    audience: str
    syllabus: Syllabus
    human_feedback: str

    lectures: Annotated[dict[str, Lecture], merge_dicts]
    verdicts: Annotated[dict[str, ReviewVerdict], merge_dicts]

    quizzes: Annotated[dict[str, Quiz], merge_dicts]
    assignments: Annotated[dict[str, Assignment], merge_dicts]

    output_dir: str
    export_bundle_path: str
```

We use a custom dictionary reducer for lectures instead of a simple list append. If a lecture gets rejected by the reviewer and rewritten, the new draft overwrites the old one by its lesson ID instead of piling up duplicates in the state. This keeps the final payload clean for the downstream production step.

The syllabus holds a flat list of lessons with explicit module ID back-references. That flat shape makes the parallel fan-out a simple one-liner later on.

1. The Curriculum Agent

The Curriculum agent is the one part most first-attempt

course generators get least wrong but most superficial. We need to do it carefully.

A naive prompt asks the model to write a syllabus with twelve lessons. Our Curriculum agent derives measurable learning objectives first. It groups them into modules and then picks lessons that teach them. This enforces the Quality Matters Rubric alignment standards mechanically.

Every learning objective carries a Bloom's Taxonomy level tag. The options are Remember, Understand, Apply, Analyze, Evaluate, and Create. This forces the Assessment agent later to produce a real mix of questions instead of ten multiple-choice vocabulary checks.

We enforce hard caps in the prompt. We limit the course to three modules and fifteen lessons total. Anything more and the system starts generating slop to fill space.

After the syllabus is drafted, the graph pauses. The human reviews the syllabus and either approves it or requests changes.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langgraph.types import Command, interrupt

CURRICULUM_PROMPT = """You are a Curriculum Architect. Giv
```

produce a full course syllabus designed *backwards from le

Hard constraints:

- 3 modules total. 10 to 15 lessons total. Do not exceed.
- Every learning objective is measurable and learner-cente
- Every module has at least one objective at Apply level o
- Every lesson maps to 1 to 3 objectives from its module.
- Prerequisites surfaced explicitly at course level.

Topic: {topic}

Audience: {audience}

Prior reviewer feedback (apply it verbatim, do not argue w
{feedback}

"""

```
def curriculum_node(state: CourseState) -> Command:
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro", t
    architect = llm.with_structured_output(Syllabus)

    syllabus: Syllabus = architect.invoke([
        {
            "role": "system",
            "content": CURRICULUM_PROMPT.format(
                topic=state["topic"],
                audience=state.get("audience", "working so
                feedback=state.get("human_feedback") or "n
            ),
        }
    ])

    decision = interrupt({
        "phase": "syllabus_review",
        "topic": state["topic"],
        "syllabus": syllabus.model_dump(),
        "options": ["approve", "revise"],
    })

    if decision.get("action") == "approve":
        return Command(
            update={"syllabus": syllabus, "human_feedback"
            goto="fan_out",
        )

    return Command(
        update={"human_feedback": decision.get("feedback",
        goto="curriculum",
    )
```

The `interrupt()` call pauses the graph. It returns whatever value the caller passes when resuming. The payload shape is the contract between the graph and your user interface.

The revise path loops back to the curriculum node. The next run reads the human feedback from state and the model sees it. This is directed revision. The full state is persisted by the checkpointer so the human can take hours to review the syllabus and the graph resumes perfectly.

2. The Content Agent

The Curriculum agent produces twelve lesson outlines. The Content agent writes all twelve in parallel.

Sequential generation takes about fifteen minutes of writing. Parallel generation takes about ninety seconds. Each lesson is generated with an independent context focused on its own objectives. This removes long-context drift completely.

```
from langgraph.types import Send

def fan_out_to_content_agents(state: CourseState) -> list[
    """Conditional edge: one Send per lesson, dispatched i
    syllabus = state["syllabus"]
    objectives_by_id = {o.id: o for o in syllabus.objectiv

    return [
        Send(
            "content",
            {
                "lesson": lesson.model_dump(),
                "objectives": [
                    objectives_by_id[oid].model_dump()
                    for oid in lesson.objectives
                    if oid in objectives_by_id
                ],
                "prerequisites": lesson.prerequisites,
                "topic": state["topic"],
            },
        )
        for lesson in syllabus.lessons
    ]

def fan_out_node(state: CourseState) -> dict:
    """Identity node – exists only to be the source of the
    return {}
```

Each `Send` is a scheduled invocation of the named node with a custom payload. When `fan_out_to_content_agents` returns a list of twelve `Send` objects, `LangGraph`

dispatches twelve independent runs of `content` in the same superstep. The content node does not receive the full `CourseState`. It receives only the small dictionary inside its `Send`. This is the mechanism that removes long-context drift. Lesson seven has no knowledge of lesson three and cannot contradict it because the model cannot see it.

The outputs merge back into the global state via the `merge_dicts` reducer we defined earlier. Each run returns `{"lectures": {lesson_id: Lecture}}`, and the reducer combines the twelve single-key dicts into one `'lectures'` dict keyed by lesson ID. Parallel writes to the same state slot do not collide because every run writes under a distinct key.

The `fan_out_node` is a shim. Conditional edges in `LangGraph` must attach to a source node, and we want the graph visualization to read cleanly. Attaching the fan-out directly to `curriculum` would tangle two unrelated branches on one node — the human-approval revise loop and the parallel writing phase — so we pay for a no-op node to keep the diagram legible.

3: The Anti-slop Reviewer

This is the part every demo skips. If you take one concept from this post, it should be this layer.

Slop includes generic filler phrases like “in today’s ever-evolving landscape” or “it’s worth noting that”. It includes buzzwords like “game-changer” or “seamless”. It lacks specificity. There are no numbers, no named tools, and no code snippets. The sentences all have the same length and conclusions are vague phrases like “only time will tell”.

Our reviewer is a structured-output agent using Gemini 2.5 Flash. It has one job. It takes a generated lecture and returns a structured verdict. It does not judge quality in

the abstract. It pattern-matches against a concrete checklist.

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langgraph.types import Command

MAX_REWRITES = 2

LESSON_PROMPT = """Write one lesson for a course on: {topic}

Lesson title: {title}
Learning objectives (must all be addressed, tagged with BL
{objectives}

Prerequisites the reader already has: {prereqs}

Skeleton (use these exact H2 headings):
## Hook
## Concept
## Mechanism
## Worked example
## Common failure
## Summary
## Checklist

Hard rules:
- 800 to 1500 words.
- At least two concrete, named artifacts (tools, libraries
- At least one code snippet OR one concrete numerical exam
- No filler phrases ("in today's evolving landscape", "it'
- No vague closers ("the possibilities are endless").
- Reviewer corrections to apply verbatim (if any): {correc
"""

def content_node(state: dict) -> dict:
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro", t
    writer = llm.with_structured_output(Lecture)

    corrections = state.get("reviewer_corrections") or "no
    rewrite_count = state.get("rewrite_count", 0)

    lecture: Lecture = writer.invoke([
        {
            "role": "system",
            "content": LESSON_PROMPT.format(
```

```

        topic=state["topic"],
        title=state["lesson"]["title"],
        objectives="\n".join(
            f"- [{o['bloom']}] {o['text']}" for o
        ),
        prereqs=", ".join(state.get("prerequisites")),
        corrections=corrections,
    ),
}
])
lecture.lesson_id = state["lesson"]["id"]
lecture.rewrite_count = rewrite_count

return {"lectures": {lecture.lesson_id: lecture}}

SLOP_PATTERNS = [
    "in today's", "ever-evolving", "it's worth noting", "i",
    "game-changer", "revolutionary", "seamless", "cutting-",
    "the possibilities are endless", "only time will tell",
    "unlock the power", "at the end of the day",
]

REVIEWER_PROMPT = """You are an anti-slop reviewer. You do
in the abstract. You pattern-match against a concrete chec
structured issues.

Reject the lecture if ANY of these are true:
- Contains filler phrases (see regex hits below).
- Uses buzzwords without concrete referents.
- Any H2 section lacks a specific named tool, library, pap
- Ends with a vague conclusion ("endless", "only time will
- Any phrase repeats 3+ times across the lecture.

Regex pre-check hits: {regex_hits}

Lecture id: {lesson_id}
Title: {title}

Return a ReviewVerdict. Approve only if clean."""

def reviewer_node(state: dict) -> Command:
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash",
    judge = llm.with_structured_output(ReviewVerdict)

    lesson_id = state["lesson"]["id"]
    lecture = state["lectures"][lesson_id]

    lowered = lecture.body_markdown.lower()
    regex_hits = [p for p in SLOP_PATTERNS if p in lowered

```



```

verdict: ReviewVerdict = judge.invoke([
    {
        "role": "system",
        "content": REVIEWER_PROMPT.format(
            regex_hits=regex_hits or "none",
            lesson_id=lecture.lesson_id,
            title=lecture.title,
        ),
    },
    {"role": "user", "content": lecture.body_markdown}
])
verdict.lesson_id = lecture.lesson_id

if verdict.approved or lecture.rewrite_count >= MAX_RE
    return Command(update={"verdicts": {lesson_id: ver

corrections = "\n".join(f"- [{i.category}] {i.fix}" fo
return Command(
    update={
        "rewrite_count": lecture.rewrite_count + 1,
        "reviewer_corrections": corrections,
    },
    goto="content",
)

def collect_node(state: CourseState) -> dict:
    return {}

```

The regex pre-check is deliberately simple. It does a literal substring match for known slop phrases. This saves the language model from having to re-derive what slop is on every call. It is a cheap prior for an expensive model.

We use a hard cap on rewrites. The `rewrite_count >= MAX_REWRITES` branch is the fallback. If the model fails twice, we record the verdict and move on. A human

editor can see which lessons the reviewer gave up on later.

The loop is local to each fan-out branch. Lesson three can be on its second rewrite while lesson seven is on its first. They all converge independently in the same superstep.

The collect node is an identity node. It acts as a join point where the twelve parallel branches rejoin before the assessment runs.

4: The Assessment Agent

The Assessment agent is structurally identical to the Content agent. It takes the module context and generates

quizzes and assignments.

The prompt asks the agent to tag each question with its Bloom level and target a specific distribution (20% Remember, 40% Apply, 30% Analyze, and 10% Evaluate). In a production system, you would add a post-generation validator here to reject the output if the ratio skews too far from the target or if a question maps to a missing objective ID.

We also generate one multi-part assignment per module scoped to the objectives at the Apply level or higher. It includes a rubric the instructor can grade against.

```
def assessment_node(state: CourseState) -> dict:
    llm = ChatGoogleGenerativeAI(model="gemini-2.5-pro", t
    quiz_writer = llm.with_structured_output(Quiz)
    brief_writer = llm.with_structured_output(Assignment)

    quizzes: dict[str, Quiz] = {}
    assignments: dict[str, Assignment] = {}

    for module in state["syllabus"].modules:
        module_objectives = [
            o for o in state["syllabus"].objectives if o.m
        ]

        quiz = quiz_writer.invoke(
            f"Write a 10-question quiz for module '{module'
            f"Distribute Bloom levels: 20% remember/unders
            f"30% analyze, 10% evaluate/create. Each quest
            f"one objective id from: {[o.id for o in modul
        )
        quizzes[module.id] = quiz

        if any(o.bloom in ("apply", "analyze", "create") f
            assignment = brief_writer.invoke(
```

```
        f"Write one multi-part assignment for module {module.id}"
        f"scoped to objectives at Apply or higher."
    )
    assignments[module.id] = assignment

    return {"quizzes": quizzes, "assignments": assignments}
```

5: Deterministic Production with Marp

This is the only part of the pipeline that is strictly LLM-optional. Slide assembly is deterministic.

We use the Marp CLI. It takes structured Markdown and produces PDF, PPTX, or HTML decks. The agent produces the Marp Markdown deterministically from each lecture's Hook, Summary, and Checklist blocks. There is no language model generation of slide content.

LLM-generated slide layouts are the single largest source of broken decks. They produce oversized text, overlapping images, and bullet mudslides. A template and deterministic assembly removes this failure category completely.

```
import re
import subprocess
from pathlib import Path

MARP_HEADER = """----
marp: true
theme: default
paginate: true
---
```

```

# {course_title}
#### {audience}

---
"""

SECTION_RE = re.compile(r"^## (Hook|Summary|Checklist)\s*\n"
                        re.MULTILINE | re.DOTALL)

def lecture_to_slides_md(lecture: Lecture, module_title: str)
    """Deterministic: no LLM call. Extract fixed sections,
    sections = {m.group(1): m.group(2).strip() for m in SE

    slides = [
        f"## {module_title}\n## {lecture.title}",
        f"### Why this lesson\n\n{sections.get('Hook', '')}
        f"### The idea\n\n{sections.get('Summary', '')}.str
    ]

    checklist = sections.get("Checklist", "").strip()
    if checklist:
        slides.append(f"### Checklist\n\n{checklist}")

    return "\n\n---\n\n".join(slides) + "\n\n---\n\n"

def build_course_deck(
    syllabus: Syllabus,
    lectures: dict[str, Lecture],
    out_dir: Path,
) -> Path:
    out_dir.mkdir(parents=True, exist_ok=True)
    md_path = out_dir / "course.md"
    pdf_path = out_dir / "course.pdf"

    modules_by_id = {m.id: m for m in syllabus.modules}

    parts = [MARF_HEADER.format(course_title=syllabus.topi
                                audience=syllabus.audienc
    for lesson in syllabus.lessons:
        lecture = lectures.get(lesson.id)
        if lecture is None:
            continue
        module_title = modules_by_id[lesson.module_id].tit
        parts.append(lecture_to_slides_md(lecture, module_

    md_path.write_text("\n".join(parts), encoding="utf-8")

    subprocess.run(

```

```
        ["marp", str(md_path), "--pdf", "-o", str(pdf_path),
         check=True,
        )
    return pdf_path

def production_node(state: CourseState) -> dict:
    out_dir = Path(state.get("output_dir") or "./course")
    pdf = build_course_deck(state["syllabus"], state["lect
    return {"export_bundle_path": str(pdf)}
```

The regex relies on the exact H2 headings imposed by the prompt earlier. The prompt guarantees the structure the downstream parser depends on. This is a crucial design pattern. Output is deterministic enough when the downstream consumer is templated.

Wiring the Full Graph

We have five nodes, one reducer-driven fan-out, and one human approval gate. We compile them into one graph.

```
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command

def build_course_graph(checkpointer=None) -> StateGraph:
    g = StateGraph(CourseState)
    g.add_node("curriculum", curriculum_node)
    g.add_node("fan_out", fan_out_node)
    g.add_node("content", content_node)
    g.add_node("reviewer", reviewer_node)
    g.add_node("collect", collect_node)
    g.add_node("assessment", assessment_node)
    g.add_node("production", production_node)

    g.add_edge(START, "curriculum")
    g.add_conditional_edges("fan_out", fan_out_to_content_)
    g.add_edge("content", "reviewer")
    g.add_edge("collect", "assessment")
    g.add_edge("assessment", "production")
    g.add_edge("production", END)

    return g.compile(checkpointer=checkpointer or MemorySaver)

def run_course_pipeline(topic: str, audience: str, thread_id: str):
    graph = build_course_graph()
    config = {"configurable": {"thread_id": thread_id}}

    result = graph.invoke(
        {"topic": topic, "audience": audience, "output_dir": output_dir},
        config=config,
    )

    while "__interrupt__" in result:
        payload = result["__interrupt__"][0].value
        print(f"\n=== Review syllabus for: {payload['topic']}")
        for mod in payload["syllabus"]["modules"]:
            print(f"  Module: {mod['title']} - {len(mod['content'])} words")

        choice = input("\n[a]pprove / [r]evise: ").strip()
        if choice.startswith("r"):
            feedback = input("What should change? ")
            return graph.invoke(
                {"topic": topic, "audience": audience, "output_dir": output_dir},
                config=config,
            )
```

```
        result = graph.invoke(
            Command(resume={"action": "revise", "feedback": config},
                    config=config,
                )
        )
    else:
        result = graph.invoke(
            Command(resume={"action": "approve"},
                    config=config,
                )
        )

    return result["export_bundle_path"]

if __name__ == "__main__":
    path = run_course_pipeline(
        topic="Teach me how to build production-grade retraining pipelines for AI engineers",
        audience="working AI engineers who have shipped a product",
        thread_id="course-2026-04-19",
    )
    print(f"\nDone. Course PDF at: {path}")
```

The edge list shows the whole architecture clearly. The transitions from curriculum to fan-out and from reviewer to collect are driven by the `Command(goto=...)` instructions inside the nodes. This is the modern LangGraph style.

The `while "__interrupt__"` in `result` loop handles the human approval contract. If the syllabus is revised three times, the loop runs three times. The thread ID keeps the state pinned across all of them.

Closing

If you want to keep building after the basic pipeline runs, you can pipe the lecture scripts into a text-to-speech API and pair it with FFmpeg for narrated MP4

exports. You can also add an alignment auditor — a second reviewer that checks whether assessments actually measure the stated objectives — which is worth doing if the output is going in front of accredited institutions.

Course creation is not replaced by this pipeline. The agent does not replace the course creator. It replaces the first three weeks of the course creator's timeline.

Continue Reading

LangGraph AI Agents Explained — The state-machine foundations behind the `StateGraph`, `reducers`, and `Command(goto=...)` patterns this pipeline relies on.

Artificial Intelligence

Machine Learning

Data Science

Software Engineering

Programming



Published in Data Science Collective

922K followers · Last published 2 days ago

Follow

Advice, insights, and ideas from the Medium data science community



Written by Anubhav

911 followers · 28 following

Follow

I write deep-dives on agents, RAG, evals, and what actually breaks when you ship AI systems

